Finger Control Robotic Arm

Finger Control Robotic Arm

1. Content Description
2. Program Startup
3. Core Source Code Analysis

1. Content Description

This function implements acquiring color images and using the mediapipe framework to detect fingers, calculating the angle between the thumb and the index finger to control the opening and closing of the robotic arm gripper (servo No. 6).

2. Program Startup

```
ros2 launch yahboomcar_bringup mediapipe_demo.launch.py demo:=FingerCtrlGripper
```

After the program runs, as shown in the figure below, once the program detects a palm, it will calculate the angle between the thumb and the index finger, slowly open and close the two fingers, and the robotic arm gripper will follow accordingly.



3. Core Source Code Analysis

Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/mediapipe/17_FingerCtrlGripper.py
```

Import the required library files,

```
import math
import time
import cv2 as cv
import numpy as np
#Import the mediapipe library
import mediapipe as mp
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints,ArmJoint
import cv2
```

Initialize the data and define the publisher and subscriber,

```
def __init__(self, name):
    super().__init__(name)
    self.lmList = []
    #Use the class in the mediapipe library to define a hand object
    self.mpHand = mp.solutions.hands
    self.mpDraw = mp.solutions.drawing_utils
    self.hands = self.mpHand.Hands(
        static_image_mode=False,
        max_num_hands=2,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5
    )

    self.rgb_bridge = CvBridge()
    #Define the topic publisher for controlling the 6 servos, and publish the detected pose
    self.TargetAngle_pub = self.create_publisher(ArmJoints, "arm6_joints", 10)
    #Define the topic publisher for controlling a single servo, to later control servo No. 6 (the gripper) individually
    self.pub_SingleTargetAngle = self.create_publisher(ArmJoint, "arm_joint", 10)
    self.init_joints = [90, 150, 10, 20, 90, 180]
    self.pubSix_Arm(self.init_joints)
    #Define the Subscriber for the color image topic
    self.sub_rgb =
self.create_subscription(Image,"/camera/color/image_raw",self.get_RGBImageCallBack,100)
```

Color image callback function,

```
def get_RGBImageCallBack(self,msg):
    #Use CvBridge to convert the color image message data into image data
    frame = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Pass the obtained image into the custom findHands function to check the palm
    img = self.findHands(frame)
    #Run the custom findPosition function to obtain the xy coordinates of the finger joints
    lmList =  self.findPosition(frame, draw=False)
    if len(lmList) != 0:
        #Calculate the angle formed by the thumb tip, wrist joint, and index finger tip
        angle =  self.calc_angle(4, 0, 8)
        print("angle: ",angle)
        #If the angle is less than 2, assign the value 2 to angle, because the maximum servo
value is 180
        if angle<2:
            angle = 2
        #Calculate the value of servo No. 6
        grasp = 360/angle
        #Execute the topic function that publishes the single servo angle
        self.pubSingleArm(6,int(grasp))
    cv.imshow('dst', frame)
    action = cv2.waitKey(1)
```

findPosition function, to obtain the xy coordinates of the finger joints

```
def findPosition(self, frame, draw=True):
    #Define a list to store the id of each joint and the xy coordinates corresponding to the
id
    self.lmList = []
    if self.results.multi_hand_landmarks:
        #Iterate through the palm detection results to obtain the joint id and the xy
coordinates corresponding to the joint id
        for id, lm in enumerate(self.results.multi_hand_landmarks[0].landmark):
            # print(id,lm)
            h, w, c = frame.shape
            cx, cy = int(lm.x * w), int(lm.y * h)
            # print(id, lm.x, lm.y, lm.z)
            self.lmList.append([id, cx, cy])
    return self.lmLis
```

calc_angle function, to calculate the angle formed by 3 points,

```
def calc_angle(self, pt1, pt2, pt3):
    #Extract the xy coordinates from the list according to the joint id
    point1 = self.lmList[pt1][1], self.lmList[pt1][2]
    point2 = self.lmList[pt2][1], self.lmList[pt2][2]
    point3 = self.lmList[pt3][1], self.lmList[pt3][2]
    #Calculate the distance between the joints
    a = self.get_dist(point1, point2)
    b = self.get_dist(point2, point3)
    c = self.get_dist(point1, point3)
    try:
```

```
        radian = math.acos((math.pow(a, 2) + math.pow(b, 2) - math.pow(c, 2)) / (2 * a *
b))
        angle = radian / math.pi * 180
    except:
        angle = 0
    return abs(angle)
```